DBMS PROJECT REPORT

Cricket Player Performance & Selection Intelligence System

DBMS Experiential Learning – Level 3 Project

Student	SRN	Role in project synopsis
P. Jaya Surya	PES1UG24AM210	SQL schema design and backend API
P. S. Sai Sreyaesh	PES1UG24AM189	Vector DB (proposed role) and Next.js frontend

Prepared to match the nine sections specified by the course report instructions

This report uses available project documentation and code details. Verify live schema, exact folder tree, screenshots and official rubric before submission.

1. Project Title

Cricket Player Performance & Selection Intelligence System

1.1 Problem statement

Cricket teams need to evaluate players across formats, venues, opponents and match conditions. Broad career totals alone may not show context-specific performance. This project organises detailed cricket information so analysts can query and compare player performance in more useful contexts.

1.2 Objectives

• Design a normalized relational database for players, teams, matches, venues, innings and performance records.

• Support multi-dimensional analysis by format, opponent, venue, match phase and weather context.

• Store flexible scouting observations in a document-oriented NoSQL database.

• Provide a web dashboard for player information, selection-oriented reports and batting analytics.

• Demonstrate DBMS concepts such as keys, constraints, joins, views, procedures, triggers and indexes.

1.3 Scope

The data model focuses on men's international cricket in ODI and T20I formats. It covers player, match, innings, batting, bowling, dismissal, opponent, venue and weather information. The project is an analytics and selection-support application, not a live-streaming, fantasy-league or betting application.

2. Student SRNs and Names

Student name	SRN	Role recorded in synopsis
P. Jaya Surya	PES1UG24AM210	SQL schema design and backend API
P. S. Sai Sreyaesh	PES1UG24AM189	Vector DB (proposed) and Next.js frontend

Confirm spelling, SRNs and role descriptions with both team members and official course records before final submission.

3. Abstract

The Cricket Player Performance & Selection Intelligence System is a database-driven web application designed to organise cricket statistics and support data-informed player evaluation. It stores information about players, teams, matches, venues, weather, innings, batting performance, bowling performance, dismissals, opponents, player roles and application users.

MySQL is used as the primary relational database for structured cricket records. The schema uses primary keys, foreign keys, unique constraints, CHECK constraints and indexes to support data integrity and efficient querying. The database also contains two views, a stored procedure named **GetPlayerBattingStats** and a BEFORE INSERT trigger named **trg_validate_batting_performance**.

MongoDB is used as a document-oriented NoSQL database for flexible scouting reports in the **scouting_reports** collection. The web application is developed using Next.js, React and TypeScript. API routes connect application functionality to the databases, while authentication and role-based authorization help protect restricted operations.

The application includes player information, player-selection views, a top batters report and advanced SQL reporting. The objective is to make cricket performance data easier to organise, query and interpret for selection-related analysis.

The original synopsis proposed a vector database/semantic similarity layer. Available implementation information confirms MySQL + MongoDB; it does not confirm a working vector database. Do not claim vector search is implemented unless you can demonstrate it.

4. Technology Stack

Technology	Purpose
Next.js	React-based web application framework; application pages and API routes.
React	Builds the user interface using reusable components.
TypeScript	Adds static types to JavaScript application code.
Tailwind CSS	Utility-first frontend styling.
MySQL	Relational database for structured cricket and account data.
MongoDB	Document-oriented database for flexible scouting reports.

Technology	Purpose
Node.js	JavaScript runtime for server-side application code.
mysql2/promise	Promise-based Node.js driver for MySQL.
MongoDB Node.js driver	Connects application code to MongoDB.
jose / JWT	Creates and verifies signed session tokens.
bcrypt	Compares submitted passwords with stored password hashes.

4.1 Database names

System	Database	Objects
MySQL	cricket_selection	12 relational tables and 2 views
MongoDB	cricket_selection_nosql	scouting_reports collection

4.2 Application architecture

The browser renders the Next.js/React interface. The frontend sends requests to Next.js API routes. Server-side routes check authentication/authorization where required, execute database operations through the relevant driver, and return JSON responses. MySQL handles structured relational records; MongoDB handles flexible scouting documents.

Application data flow

```
Browser / React UI
        |
        | HTTP request (fetch)
        v
Next.js API route
        |
        +---- mysql2/promise ----> MySQL: cricket_selection
        |
        +---- MongoDB driver ---> MongoDB: cricket_selection_nosql
        |
        v
JSON response to frontend
```

5. SQL – Database Commands

The commands in this section are for MySQL. Select the project database first. Read-only commands are safe to inspect; INSERT, UPDATE, DELETE, ALTER and DROP must be used carefully.

5.1 Select database and inspect tables

```
USE cricket_selection;
SHOW TABLES;
DESCRIBE player;
DESCRIBE `match`;
DESCRIBE innings;
DESCRIBE batting_performance;
DESCRIBE bowling_performance;
SHOW CREATE TABLE player;
SHOW CREATE TABLE batting_performance;
```

DESCRIBE displays columns, data types, NULL rules, keys and defaults. SHOW CREATE TABLE displays the table definition and constraints.

5.2 DDL – Data Definition Language

DDL defines or changes database structures.

```
CREATE TABLE demo_table (
    id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

ALTER TABLE demo_table
ADD COLUMN country VARCHAR(50);

-- Remove only this disposable demo table
DROP TABLE demo_table;
```

Use the separate demo_table only. Do not DROP or ALTER project tables for a classroom demonstration.

5.3 DML – Data Manipulation Language

DML inserts, modifies and deletes records. The PLAYER table requires player_name, country and batting_style. bowling_style and date_of_birth are nullable; player_id is auto-increment.

```
INSERT INTO player
    (player_name, country, batting_style)
```

```
VALUES
    ('Demo Player 2026', 'India', 'Right hand bat');

SELECT *
FROM player
WHERE player_name = 'Demo Player 2026';

UPDATE player
SET country = 'India'
WHERE player_name = 'Demo Player 2026';

-- Delete only the test record after verifying it
DELETE FROM player
WHERE player_name = 'Demo Player 2026';
```

Use a unique demo name. Verify the WHERE condition before UPDATE or DELETE.

5.4 SELECT, WHERE, DISTINCT, ORDER BY, LIMIT

```
SELECT player_id, player_name, country
FROM player;

SELECT player_name, country
FROM player
WHERE country = 'India';

SELECT DISTINCT country FROM player;

SELECT player_name
FROM player
ORDER BY player_name ASC;

SELECT * FROM batting_performance LIMIT 10;

SELECT player_name
FROM player
WHERE bowling_style IS NULL;
```

5.5 Aggregate functions

COUNT counts rows/values, SUM adds values, AVG calculates an average, MAX returns the largest value and MIN returns the smallest value.

```
SELECT COUNT(*) AS total_batting_records
FROM batting_performance;

SELECT SUM(runs_scored) AS total_runs
FROM batting_performance;

SELECT AVG(runs_scored) AS average_runs_per_record
FROM batting_performance;

SELECT MAX(runs_scored) AS highest_record_score
FROM batting_performance;

SELECT MIN(runs_scored) AS lowest_record_score
FROM batting_performance;
```

5.6 GROUP BY and HAVING

```
SELECT p.player_id, p.player_name,
       SUM(bp.runs_scored) AS total_runs
FROM player p
JOIN batting_performance bp
  ON p.player_id = bp.player_id
GROUP BY p.player_id, p.player_name
ORDER BY total_runs DESC;

SELECT p.player_id, p.player_name,
       SUM(bp.runs_scored) AS total_runs
FROM player p
JOIN batting_performance bp
  ON p.player_id = bp.player_id
GROUP BY p.player_id, p.player_name
HAVING SUM(bp.runs_scored) > 1000;
```

GROUP BY groups rows sharing a value. HAVING filters groups after aggregation; WHERE filters individual rows before grouping.

5.7 JOIN commands

JOIN combines rows from related tables. PLAYER.player_id links to batting_performance.player_id; innings_id links performance records with INNINGS.

INNER JOIN – matching rows only

```
SELECT p.player_name, bp.runs_scored
FROM player p
INNER JOIN batting_performance bp
  ON p.player_id = bp.player_id;
```

Use it when only matching records from both tables are needed. JOIN without a type means INNER JOIN in MySQL.

LEFT JOIN – keep all left-side rows

```
SELECT p.player_name, bp.runs_scored
FROM player p
LEFT JOIN batting_performance bp
  ON p.player_id = bp.player_id;
```

Retains every player, even without a batting record. Unmatched batting columns are NULL.

RIGHT JOIN – keep all right-side rows

```
SELECT p.player_name, bp.runs_scored
FROM player p
RIGHT JOIN batting_performance bp
  ON p.player_id = bp.player_id;
```

Retains every batting performance row and matching player details.

FULL OUTER JOIN – all rows from both sides

MySQL does not directly support FULL OUTER JOIN. A LEFT JOIN and RIGHT JOIN can be combined with UNION.

```
SELECT p.player_name, bp.runs_scored
FROM player p
LEFT JOIN batting_performance bp
  ON p.player_id = bp.player_id
UNION
SELECT p.player_name, bp.runs_scored
FROM player p
RIGHT JOIN batting_performance bp
  ON p.player_id = bp.player_id;
```

CROSS JOIN – Cartesian product

```
SELECT p.player_name, t.team_name
FROM player p
CROSS JOIN team t;
```

Pairs every player with every team. If the tables contain P and T rows, respectively, the result contains P × T combinations.

SELF JOIN – table joined to itself

```
SELECT p1.player_name AS player_one,
       p2.player_name AS player_two,
       p1.country
FROM player p1
JOIN player p2
  ON p1.country = p2.country
 AND p1.player_id < p2.player_id;
```

Used to compare rows in the same table. p1 and p2 are aliases for PLAYER. The ID comparison avoids duplicate pairs and self-pairs.

Multi-table JOIN

```
SELECT p.player_name, i.innings_number,
       bp.runs_scored, bp.balls_faced
FROM player p
JOIN batting_performance bp
  ON p.player_id = bp.player_id
JOIN innings i
  ON bp.innings_id = i.innings_id
ORDER BY p.player_name;
```

Combines player, batting performance and innings details in one result.

5.8 Nested queries / subqueries

A subquery is a SELECT statement placed inside another SQL statement. The outer query uses the inner query's result.

Single-value subquery

```
SELECT player_id, runs_scored
FROM batting_performance
WHERE runs_scored > (
    SELECT AVG(runs_scored)
    FROM batting_performance
);
```

IN with a multi-row subquery

```
SELECT player_name
FROM player
WHERE player_id IN (
    SELECT player_id
    FROM batting_performance
);
```

EXISTS and NOT EXISTS

```
-- Players with at least one batting record
SELECT p.player_name
FROM player p
WHERE EXISTS (
```

```
    SELECT 1 FROM batting_performance bp
    WHERE bp.player_id = p.player_id
);

-- Players without a batting record
SELECT p.player_name
FROM player p
WHERE NOT EXISTS (
    SELECT 1 FROM batting_performance bp
    WHERE bp.player_id = p.player_id
);
```

ANY and ALL

```
-- True if comparison holds for at least one value
SELECT *
FROM batting_performance
WHERE runs_scored > ANY (
    SELECT MAX(runs_scored)
    FROM batting_performance
    GROUP BY player_id
);

-- True if comparison holds for every returned value
SELECT *
FROM batting_performance
WHERE runs_scored > ALL (
    SELECT MAX(runs_scored)
    FROM batting_performance
    GROUP BY player_id
);
```

IN checks membership in a returned list. EXISTS checks whether at least one row is returned. NOT EXISTS checks that no row is returned. ANY means at least one value; ALL means every value.

5.9 Correlated subqueries

A correlated subquery refers to a column from the outer query and depends on the current outer row.

```
SELECT p.player_name
FROM player p
WHERE EXISTS (
    SELECT 1
    FROM batting_performance bp
    WHERE bp.player_id = p.player_id
      AND bp.runs_scored > 100
);
```

The expression bp.player_id = p.player_id references the outer query alias p, making this a correlated subquery.

```
SELECT p.player_name
FROM player p
WHERE (
    SELECT MAX(bp.runs_scored)
    FROM batting_performance bp
    WHERE bp.player_id = p.player_id
) > 100;
```

A correlated subquery is a type of subquery. An independent subquery can execute without outer-row values; a correlated subquery uses values from the outer query.

5.10 Views

A view is a named query that behaves like a virtual table. The project documents two views.

```
SHOW FULL TABLES
WHERE Table_type = 'VIEW';

SELECT * FROM vw_player_details LIMIT 10;
SELECT * FROM vw_batting_performance LIMIT 10;

SHOW CREATE VIEW vw_player_details;
SHOW CREATE VIEW vw_batting_performance;
```

vw_player_details presents player profile fields. vw_batting_performance joins player, batting performance and innings information for reporting.

5.11 Stored procedure

GetPlayerBattingStats accepts a player-name parameter and retrieves matching player details and batting statistics, including batting record count, total runs, fours and sixes.

```
SHOW PROCEDURE STATUS
WHERE Db = 'cricket_selection';

SHOW CREATE PROCEDURE GetPlayerBattingStats;

CALL GetPlayerBattingStats('Virat Kohli');
CALL GetPlayerBattingStats('Unknown Player');
```

A stored procedure is a reusable SQL program stored in the database. CALL executes it with the supplied parameter.

5.12 Trigger

trg_validate_batting_performance is a BEFORE INSERT trigger on batting_performance. It checks runs_scored, balls_faced, fours and sixes. If any is negative, it raises SQLSTATE 45000 with message 'Batting statistics cannot be negative'.

```
SHOW TRIGGERS;
SHOW CREATE TRIGGER trg_validate_batting_performance;
```

The documented trigger runs on INSERT only. Do not say it validates UPDATE operations unless a separate UPDATE trigger or constraint is verified.

5.13 Constraints and integrity checks

The schema uses primary keys, foreign keys, unique constraints and CHECK constraints. Examples include different participating teams, allowed match formats, non-negative performance statistics and innings/team consistency.

```
-- Winner must be one of the two participating teams
SELECT *
FROM `match`
WHERE winner_team_id IS NOT NULL
  AND winner_team_id NOT IN (team1_id, team2_id);

-- Innings teams must belong to the referenced match
SELECT i.*
FROM innings i
JOIN `match` m ON i.match_id = m.match_id
WHERE i.batting_team_id NOT IN (m.team1_id, m.team2_id)
   OR i.bowling_team_id NOT IN (m.team1_id, m.team2_id);
```

If these validation queries return zero rows, no violations were found for the specific rule checked at that time.

5.14 Indexes and EXPLAIN

Composite index	Table	Columns
idx_batting_player_innings	batting_performance	(player_id, innings_id)
idx_bowling_player_innings	bowling_performance	(player_id, innings_id)
idx_match_venue_format	match	(venue_id, match_format)
idx_opponent_player_team	opponent	(player_id, team_id)

```
SHOW INDEX FROM batting_performance;
SHOW INDEX FROM bowling_performance;
SHOW INDEX FROM `match`;
SHOW INDEX FROM opponent;

EXPLAIN
SELECT *
FROM batting_performance
WHERE player_id = 4405
  AND innings_id = 1;
```

Indexes can improve selected lookups and joins. EXPLAIN displays the query execution plan. The sample IDs are illustrative; use IDs that exist in the live database.

5.15 Transactions and ACID

```
START TRANSACTION;

UPDATE player
SET country = 'Test Country'
WHERE player_id = 4405;

SELECT player_id, player_name, country
FROM player
WHERE player_id = 4405;

ROLLBACK;
```

A transaction groups operations into a unit of work. COMMIT saves uncommitted changes; ROLLBACK cancels them. Use a known test record and do not change important production data.

ACID property	Meaning
Atomicity	All operations succeed as a unit or are rolled back.
Consistency	A transaction preserves defined database rules.
Isolation	Concurrent transactions are controlled to avoid improper interference.
Durability	Committed changes persist.

5.16 CTE and window function

The dashboard's Advanced SQL Reports feature is documented as using a CTE, JOIN, aggregation and RANK. The following is a representative query that ranks players by total runs.

```
WITH PlayerTotals AS (
    SELECT p.player_id, p.player_name, p.country,
           SUM(bp.runs_scored) AS total_runs
    FROM player p
    JOIN batting_performance bp
      ON p.player_id = bp.player_id
    GROUP BY p.player_id, p.player_name, p.country
)
SELECT player_name, country, total_runs,
       RANK() OVER (ORDER BY total_runs DESC) AS player_rank
FROM PlayerTotals
ORDER BY player_rank;
```

A CTE is a named temporary result set defined with WITH. RANK assigns ranks by total runs; tied values share a rank and the next rank may be skipped.

6. NoSQL / Vector Database Commands

6.1 MongoDB implementation

The project documentation identifies MongoDB database **cricket_selection_nosql** and collection **scouting_reports**. MongoDB stores documents rather than fixed relational rows. This is useful for scouting notes and assessments whose attributes may vary.

Vector database status: the original synopsis proposed semantic/vector search, but the implementation evidence available for this report confirms MongoDB scouting reports only. No vector database commands are presented as implemented functionality.

6.2 Select database and inspect collection

```
use cricket_selection_nosql
show collections

db.scouting_reports.findOne()
db.scouting_reports.find().limit(5)
```

Run findOne() first to inspect the real document structure and field names before using filters or updates.

6.3 Insert a sample document

```
db.scouting_reports.insertOne({
  playerName: "Demo Player",
  recommendation: "Consider",
  notes: "Sample scouting report for demonstration"
})
```

This is an illustrative document structure. Adapt field names to the actual collection and API.

6.4 Find documents

```
db.scouting_reports.find()

db.scouting_reports.find({
  playerName: "Demo Player"
})

db.scouting_reports.findOne({
  playerName: "Demo Player"
})
```

6.5 Update a document

```
db.scouting_reports.updateOne(
  { playerName: "Demo Player" },
  { $set: { recommendation: "Selected" } }
)
```

6.6 Delete a demonstration document

```
db.scouting_reports.deleteOne({
  playerName: "Demo Player"
})
```

Only delete the unique test document you inserted. Check the filter with find() first. Never run an unrestricted deleteMany({}) against the project collection.

6.7 Aggregation pipeline

```
db.scouting_reports.aggregate([
  {
    $group: {
      _id: "$recommendation",
      totalReports: { $sum: 1 }
    }
  },
  {
```

```
    $sort: { totalReports: -1 }
  }
])
```

$group groups reports by recommendation and counts them. $sort orders the grouped results. Confirm recommendation is an actual field in the collection before running this example.

6.8 SQL and MongoDB comparison

MySQL	MongoDB
Database contains tables	Database contains collections
Table contains rows	Collection contains documents
Rows use columns	Documents use fields
Relationships commonly use foreign keys and joins	Related data may be embedded or referenced
SQL query language	MongoDB query and aggregation API

7. Next.js – Project Tree Structure

The project uses the Next.js App Router. This is a focused tree of known files from the project details reviewed; it is not a verified complete export of every local folder.

```
cricket-selection-app/
|
+-- app/
|   +-- page.tsx
|   +-- api/
|       +-- players/
|       |   +-- route.ts
|       +-- auth/
|           +-- login/
|               +-- route.ts
|
+-- lib/
|   +-- db.ts
|   +-- mongodb.ts
|   +-- auth.ts
|   +-- requireAdmin.ts
|
+-- docs/
|   +-- ER_Diagram.md
|   +-- data-dictionary.md
|
+-- ER_DIAGRAM1.png
+-- package.json
+-- .env.local  (local secrets; never publish)
```

Path	Purpose
app/page.tsx	Main dashboard; React state/effects and report components.
app/api/players/route.ts	GET API route; checks session and retrieves player fields from MySQL.
app/api/auth/login/route.ts	POST login route; verifies credentials and creates a session.
lib/db.ts	Creates and exports the MySQL connection pool.
lib/mongodb.ts	Creates/reuses MongoDB client connection and selects the project database.
lib/auth.ts	JWT session creation and verification using jose.
lib/requireAdmin.ts	Checks authenticated session and admin role for restricted operations.
docs/ER_Diagram.md	ER diagram documentation.
docs/data-dictionary.md	Database tables, fields, keys and constraints.

For final submission, run `tree /F /A` in the project root and replace/supplement this focused tree with the exact output. Never include .env.local contents or secrets.

7.1 Environment configuration

The MySQL connection reads DB_HOST, DB_PORT, DB_USER, DB_PASSWORD and DB_NAME. MongoDB uses MONGODB_URI. Authentication uses AUTH_SECRET. Keep these values in a local environment file and do not commit them to a public repository.

```
# Variable names only — never paste real secrets into the report
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=<your-local-password>
DB_NAME=cricket_selection
MONGODB_URI=<your-local-mongodb-connection-string>
AUTH_SECRET=<a-long-random-secret>
```

8. Sample Screenshots of Application Code and Demo

Insert genuine screenshots from the running project and database tools. The checklist below identifies the evidence to capture; mock screenshots should not replace actual implementation evidence.

No.	Screenshot	Where / what it proves
1	Main application dashboard	Browser; application runs.
2	Player list / selection interface	Browser; player information is displayed.
3	Top batters report	Browser; batting analytics/report.
4	Advanced SQL report	Browser; advanced report output.
5	MySQL connection pool	VS Code: lib/db.ts.
6	Players API route	VS Code: app/api/players/route.ts.
7	MongoDB connection	VS Code: lib/mongodb.ts.
8	Login and JWT code	VS Code: login route and lib/auth.ts.
9	ER diagram	Open ER_DIAGRAM1.png.
10	MySQL tables	SHOW TABLES and DESCRIBE outputs.
11	JOIN / GROUP BY output	MySQL query and result grid.
12	Stored procedure output	CALL GetPlayerBattingStats(...) and result.
13	Trigger definition	SHOW CREATE TRIGGER output.
14	Views and indexes	SHOW CREATE VIEW and SHOW INDEX output.
15	MongoDB documents	mongosh findOne() and find().limit(...).
16	MongoDB aggregation	Aggregation pipeline and result.

8.1 Screenshot quality checklist

• Show the application/database tool clearly and keep the relevant filename or query visible.

• For SQL evidence, show both the query and its successful output.

• For code evidence, capture enough surrounding lines to identify the function or route.

• Do not show passwords, AUTH_SECRET, connection credentials, tokens or other secrets.

• Use genuine outputs from the local project; do not fabricate success messages or result counts.

8.2 Suggested demonstration sequence

• Start the Next.js application and open the dashboard.

• Show the player list and one report.

• Open MySQL and show the 12 tables.

• Demonstrate one JOIN, one aggregate query and one subquery.

• Show both views and call GetPlayerBattingStats.

• Show the batting validation trigger definition.

• Show the MongoDB database, scouting_reports collection and a sample document.

• Explain how the frontend calls API routes and how the API connects to the databases.

9. Evaluation Rubrics – Completion Status

This is a cautious status summary based on project documentation and code details available when preparing the report. Compare it with the instructor's official rubric and update it only after checking the live application and database.

Requirement / evidence	Status from available information	Evidence to show
Project title, problem and objectives	Documented	Synopsis / report sections 1–3
Technology stack	Documented	package.json and source files
MySQL relational database	Implemented / documented	USE cricket_selection; SHOW TABLES
12 MySQL tables	Documented	SHOW TABLES and DESCRIBE outputs
PKs and FKs	Documented	SHOW CREATE TABLE; INFORMATION_SCHEMA
Unique and CHECK constraints	Documented	SHOW CREATE TABLE outputs

Requirement / evidence	Status from available information	Evidence to show
Four composite indexes	Documented	SHOW INDEX outputs
Two views	Documented	SHOW CREATE VIEW outputs
GetPlayerBattingStats procedure	Documented	SHOW CREATE PROCEDURE and CALL output
Batting validation trigger	Documented	SHOW CREATE TRIGGER and safe test
MongoDB scouting_reports	Implemented / documented	show collections; findOne()
Vector DB / semantic search	Not confirmed implemented	Do not mark complete without a working demo
Next.js dashboard and API routes	Implemented / documented	Browser screenshots and source code
Authentication and role checks	Implemented / documented	Login route, auth.ts, requireAdmin.ts
Screenshots and demo evidence	To be inserted	Actual screenshots from local run
Final project tree	To be verified	tree /F /A output
Final rubric sign-off	Pending official checklist	Course evaluation sheet

These labels are not an official instructor grade. Recheck the live database and application before submission.

Appendix A – Useful verification commands

A.1 MySQL object inventory

```
USE cricket_selection;
SHOW TABLES;
SHOW FULL TABLES WHERE Table_type = 'VIEW';
SHOW TRIGGERS;

SHOW PROCEDURE STATUS
WHERE Db = 'cricket_selection';

SELECT TABLE_NAME, COLUMN_NAME,
       REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'cricket_selection'
  AND REFERENCED_TABLE_NAME IS NOT NULL
ORDER BY TABLE_NAME, COLUMN_NAME;
```

A.2 MongoDB verification

```
use cricket_selection_nosql
show collections
db.scouting_reports.countDocuments()
db.scouting_reports.findOne()
db.scouting_reports.find().limit(5)
```

A.3 Next.js run commands

```
# Run from project root; confirm scripts in package.json
npm install
npm run dev
# Open the local URL printed in the terminal,
# commonly http://localhost:3000
```

Use the scripts actually listed in package.json. Confirm the project's own scripts before presenting these as exact.

Appendix B – Short viva explanations

Question	Short answer
Why MySQL?	Stores structured relational records and supports keys, constraints, joins and transactions.
Why MongoDB?	Stores flexible scouting reports as documents.
What is a primary key?	A column or set of columns that uniquely identifies a row.
What is a foreign key?	A column that references a key in another table to maintain referential integrity.
What is a view?	A named query that presents data like a virtual table.
What is a stored procedure?	A reusable SQL program stored in the database and invoked with CALL.
What is a trigger?	A database program that runs automatically in response to a specified table event.
What is an index?	A data structure that can speed up selected lookups and joins.
What is an API route?	A server endpoint that receives requests, performs logic and returns a response.
Authentication vs authorization?	Authentication verifies identity; authorization checks permitted actions.

Appendix C – Final submission checks

- Confirm both students' names, SRNs and contribution descriptions.
- Run SQL commands against the current cricket_selection database and retain real output screenshots.
- Verify live table/column names using DESCRIBE before data-changing statements.
- Inspect an actual MongoDB scouting report before using example field names.
- Capture the full project tree from the local repository.
- Insert genuine screenshots requested in Section 8.
- Check the official course evaluation rubric and update Section 9.
- Remove placeholders and ensure no secrets appear in screenshots or report files.